

Venus Labs - Dual Price Oracle

Security Assessment

CertiK Assessed on Apr 23rd, 2026





Certik Assessed on Apr 23rd, 2026

Venus Labs - Dual Price Oracle

The security assessment was prepared by Certik.

Executive Summary

TYPES

Lending, Oracle

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 04/21/2026

Final report published on 04/23/2026

Vulnerability Summary



6

Total Findings

6

Resolved

0

Partially Resolved

0

Acknowledged

0

Declined

0 Centralization

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

1 Medium

1 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

3 Minor

3 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

2 Informational

2 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS LABS - DUAL PRICE ORACLE

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Findings

[VLD-01 : Permissionless Transient Cache Warming Can Freeze Stale Oracle Prices For The Rest Of A Transaction](#)

[VLD-02 : Uninitialized Assets Receive Prices Without Governance Approval](#)

[VLD-03 : Missing Sanity Checks In Keeper Bound Updates Can Create Invalid Threshold Window](#)

[VLD-04 : Manipulated Oracle Extrema Are Persisted Before Protection Validation](#)

[VLD-05 : Inconsistent Error Descriptions In `IDeviationBoundedOracle` Interface](#)

[VLD-06 : Misleading Documentation For `\\_exceedsDeviationThreshold` Function](#)

Appendix

Disclaimer

CODEBASE | VENUS LABS - DUAL PRICE ORACLE

Repository

Changes made in <https://github.com/VenusProtocol/oracle/pull/306> (updated by pull/310) and <https://github.com/VenusProtocol/venus-protocol/pull/668>

Commit

[2519e92aeb939a3077f2e2ea735f2e5d57f8b879](#)

[7ad77af9973b7e52e587020eb8ab38ed11f5d95f](#)

[c856c36f5e920dd98804e460d14b3814d53f6f27](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | VENUS LABS - DUAL PRICE ORACLE

This audit was conducted for Venus Labs to evaluate the security and correctness of the smart contracts associated with the Venus Labs - Dual Price Oracle project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | VENUS LABS - DUAL PRICE ORACLE



6
Total Findings

0
Critical

0
Centralization

0
Major

1
Medium

3
Minor

2
Informational

This report has been prepared for Venus Labs to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 6 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VLD-01	Permissionless Transient Cache Warming Can Freeze Stale Oracle Prices For The Rest Of A Transaction	Concurrency	Medium	● Resolved
VLD-02	Uninitialized Assets Receive Prices Without Governance Approval	Volatile Code	Minor	● Resolved
VLD-03	Missing Sanity Checks In Keeper Bound Updates Can Create Invalid Threshold Window	Logical Issue	Minor	● Resolved
VLD-04	Manipulated Oracle Extrema Are Persisted Before Protection Validation	Volatile Code	Minor	● Resolved
VLD-05	Inconsistent Error Descriptions In <code>IDeviationBoundedOracle</code> Interface	Inconsistency	Informational	● Resolved
VLD-06	Misleading Documentation For <code>_exceedsDeviationThreshold</code> Function	Inconsistency	Informational	● Resolved

VLD-01 | Permissionless Transient Cache Warming Can Freeze Stale Oracle Prices For The Rest Of A Transaction

Category	Severity	Location	Status
Concurrency	● Medium	contracts/DeviationBoundedOracle.sol (oracle): 617~618; contracts/Comptroller/Diamond/facets/FacetBase.sol (protocol): 313~314	● Resolved

Description

Both `DeviationBoundedOracle` and `ResilientOracle` support permissionless intra-transaction cache warming via transient storage. This allows an attacker to pin a price snapshot before later price-affecting actions in the same transaction. If any configured `ResilientOracle` source for an asset is onchain-derived or otherwise mutable within the same transaction, subsequent bounded-price reads will use stale cached values instead of the current oracle result, breaking the intended "read latest protected price" invariant.

Impact: An attacker can call `updateProtectionState(vToken)` early in a transaction to cache bounded prices, then change the state that a configured oracle depends on, and finally execute protocol logic that consumes `getBounded*PriceView()` or the non-view bounded price path. Because both paths early-return from cache, the protocol may use outdated collateral/debt prices during security-sensitive execution. This can weaken manipulation resistance and potentially enable under-collateralized borrows, favorable liquidations, or bypass of protection triggering assumptions.

Why this is reachable:

- `DeviationBoundedOracle.updateProtectionState(address)` is permissionless and caches resolved prices for the transaction.
- Both view and non-view bounded-price paths first read transient cache and return cached values if present.
- `ResilientOracle` also exposes permissionless cache warming.
- `ResilientOracle` is not restricted to immutable offchain feeds; governance can configure onchain-derived oracle implementations (e.g., `PendleOracle`, `CorrelatedTokenOracle`).

Scenario

1. Attacker starts a transaction and calls `DeviationBoundedOracle.updateProtectionState(vToken)`.
2. This fetches spot from `RESILIENT_ORACLE`, updates bounds, resolves bounded prices, and caches them in transient storage.
3. Later in the same transaction, attacker changes the state used by one of the configured oracle sources for the asset.
4. Protocol logic calls `getBoundedCollateralPriceView`, `getBoundedDebtPriceView`, `getBoundedPricesView`, or a non-view bounded price path.
5. The oracle returns the stale cached values from step 2 rather than recomputing from the updated oracle state.

I Recommendation

Choose and enforce one of these mitigations:

- Do not use transaction cache for security-critical price reads; always recompute in the final execution path.
- Only allow transient caching for assets whose oracle stack is guaranteed not to change intra-transaction.

VLD-02 | Uninitialized Assets Receive Prices Without Governance Approval

Category	Severity	Location	Status
Volatile Code	Minor	contracts/DeviationBoundedOracle.sol (oracle): 601-605	Resolved

Description

`DeviationBoundedOracle` silently serves prices for uninitialized assets, bypassing the governance-controlled listing model. In both price paths (`_updateAndGetBoundedPrices` and `_computeBoundedPrices`), the contract reads `assetProtectionConfig[asset]` directly and returns `(spot, spot)` whenever `isBoundedPricingEnabled` is false, without first checking `_ensureInitialized(asset)`.

As a result:

1. An asset that was never initialized via `setTokenConfig()` is treated the same as an initialized-but-disabled asset and still gets a live `RESILIENT_ORACLE` price.
2. This undermines the stated "initialize protection for a new asset" flow and can expose markets that governance never intended this oracle to support.
3. It weakens the explicit `VAINotAllowed` restriction, because `_setTokenConfig()` forbids initializing VAI, but the read paths can still price VAI directly if `vToken == vai` and `RESILIENT_ORACLE` has a price for it.

Recommendation

Add a check for initialization before returning prices. In both `_updateAndGetBoundedPrices` and `_computeBoundedPrices`, verify that the asset has been initialized by checking `state.asset != address(0)` before proceeding with price calculations. This ensures that only governance-approved assets can be priced by this oracle.

Alleviation

[Venus Labs, 04/23/2026]:

Acknowledged. No changes required.

Intended behaviour. DBO is a protection layer over ResilientOracle; when an asset is not initialized or `isBoundedPricingEnabled` is false, returning spot directly is the correct fallback. Governance controls which assets receive protection via `setTokenConfig`.

VLD-03 | Missing Sanity Checks In Keeper Bound Updates Can Create Invalid Threshold Window

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DeviationBoundedOracle.sol (oracle): 375~380, 582~590	Resolved

Description

The `updateMinPrice()` and `updateMaxPrice()` functions only enforce ordering versus current spot and the opposite bound, but do not ensure the adjusted trigger bounds remain non-overlapping. As a result, a keeper can set `minPrice` and `maxPrice` such that `minPrice * (1 + triggerThreshold) < maxPrice * (1 - triggerThreshold)`, making `_exceedsDeviationThreshold()` true for any spot price and causing protection mode to activate or remain active regardless of the live price.

Additionally, the same issue can be created by governance when changing thresholds via `setThresholds(...)`, as this path also doesn't validate the invariant with current min/max prices.

Impact: This allows creation of a permanently triggered protection state where the deviation threshold condition is always satisfied, potentially locking assets in protection mode indefinitely or causing unexpected protection activation.

Recommendation

Add validation in `_validateAndUpdateBound()` to reject bound updates that would make the adjusted trigger window overlap. Also apply the same invariant check in `setThresholds()` to ensure threshold changes don't create invalid conditions with current min/max prices. The validation should ensure: `minPrice * (1 + triggerThreshold) >= maxPrice * (1 - triggerThreshold)`.

Alleviation

[Venus Labs, 04/23/2026]: Acknowledged. No changes required.

We've analyzed this and consider it a documented design property rather than a defect.

The relation `minPrice * (1 + triggerThreshold) >= maxPrice * (1 - triggerThreshold)` is not a preserved invariant of the system; it is the definition of `triggerThreshold` restated. When the observed 15-min window ratio `maxPrice / minPrice` exceeds `(1 + t) / (1 - t)`, realised volatility has by definition exceeded the risk provider's calibration for that asset, and "protection mode remains active regardless of spot" is the specified failsafe response, not a malfunction.

Adding the proposed check would actively harm the protocol:

1. Keeper mirrors the true off-chain 15-min window and is constrained by `newMin <= spot`, `newMax >= spot`. If the real window is genuinely wider than `(1+t)/(1-t)`, the invariant-breaking write is the correct write. Rejecting it would force the on-chain window to stay stale, strictly worse for manipulation resistance.

2. Governance (setThresholds) calibrates triggerThreshold from asset-level historical analysis independent of current window state. Conditioning a tightening on "current window must fit new threshold first" inverts the operational flow risk providers use.

3. The check as proposed would also be inconsistent: it only targets `_validateAndUpdateBound` and `setThresholds`, while `_expandPriceWindow`, the path that actually grows the window from noisy user observations is uncovered. A "guarded keeper / ungated user path" is the wrong shape.

Recovery from the always-triggered state is unchanged and works as designed: keeper shrinks the window under `newMin <= spot / newMax >= spot` until a safe zone exists around the current spot, cooldown elapses, `rangeRatio < resetThreshold` is reached, and `exitProtectionMode` can be called. Typical recovery: ~2 keeper ticks + one cooldown period. Keeper liveness is a documented hard dependency.

We therefore consider the behavior correct and decline the proposed fix.

Same principle as VLD-04: `KEEPER_DEADBAND = 5% <= MIN_THRESHOLD = 5% <= triggerThreshold`, so any state the keeper is responsible for correcting is always above the deadband and is always pushed on the next tick. The worst case is bounded by keeper cadence regardless of whether the window appears "invariant-breaking" or not. The two findings are the same class of concern with the same answer: the keeper is the designated recovery mechanism; the on-chain geometry is a reflection of reality, not a precondition to enforce.

VLD-04 | Manipulated Oracle Extrema Are Persisted Before Protection Validation

Category	Severity	Location	Status
Volatile Code	Minor	contracts/DeviationBoundedOracle.sol (oracle): 631-637	Resolved

Description

`DeviationBoundedOracle` persists newly observed price extrema before validating whether the current spot should trigger protection. In `_expandPriceWindow()`, if the spot price is a new minimum or maximum, it immediately updates `state.minPrice` or `state.maxPrice` before checking if protection should be triggered in `_checkAndTriggerProtection()`.

Impact: If the upstream oracle can be briefly manipulated to a non-zero extreme, the manipulated price is written into `minPrice` / `maxPrice` and can continue skewing bounded pricing across later transactions while protection is active. This poisons the stored window and delays return to normal pricing until keepers repair the bounds, even after the manipulation has ended.

Problematic flow:

1. Manipulator briefly pushes oracle price to extreme value
2. `_expandPriceWindow()` updates min/max with manipulated price
3. Protection may or may not trigger, but window is now corrupted
4. Future transactions use corrupted window for bounded pricing calculations

Recommendation

Re-order the logic to detect/protect before persisting new extrema. Consider quarantining suspicious extrema instead of committing them immediately, or implement a validation step that checks if the extreme price should be trusted before updating the stored window.

Alleviation

[Venus Labs, 04/23/2026]: Acknowledged, no changes required.

The ordering is intentional and relies on the keeper, which is a core protocol dependency. `_expandPriceWindow()` is a user-triggered, one-way mechanism for tracking new extremes. It is not the canonical source of accurate window bounds; the off-chain keeper is. Keeper updates run roughly every minute with a 5% deadband. Because corrections allow shrinking stale/poisoned extrema back toward spot ($\text{newMin} \leq \text{currentSpot}$, $\text{newMax} \geq \text{currentSpot}$), manipulated bounds are expected to be repaired on the next keeper cycle.

Reordering to "protect before persist" would prevent immediate user-triggered window expansion and move full responsibility for legitimate window growth to the keeper, creating stale bounds during real volatility. In both designs, exposure is effectively

bounded by keeper cadence. We consider that acceptable and will manage residual risk through keeper liveness/monitoring rather than changing protocol logic.

VLD-05 | Inconsistent Error Descriptions In `IDeviationBoundedOracle` Interface

Category	Severity	Location	Status
Inconsistency	● Informational	contracts/interfaces/IDeviationBoundedOracle.sol (oracle): 74~75, 110~111, 116~117, 158~159	● Resolved

Description

The `IDeviationBoundedOracle` interface contains multiple error descriptions that are either misleading or inconsistent with the actual implementation:

1. `MarketNotInitialized`: Currently described as "Thrown when trying to initialize protection for an asset that is not initialized" - but this error is actually thrown when trying to use/update protection for an asset that hasn't been initialized via `setTokenConfig()`.
2. `ProtectedPriceActive`: Described as "when trying to update for an asset with active protection" - but specifically refers to disabling bounded pricing while protection is active in `setAssetBoundedPricingEnabled()`.
3. `InvalidResetThreshold`: Comment says "set above the deviation threshold" - but the actual check is `>= triggerThreshold` ("at or above").
4. `updateProtectionState`: Described as "caching the resolved price" - but actually caches both collateral and debt prices.

These inconsistencies create confusion for developers and auditors reviewing the interface documentation.

Recommendation

Update all error and function descriptions in the interface to accurately match their implementation:

- `MarketNotInitialized`: "Thrown when trying to use or update protection for an asset that has not been initialized"
- `ProtectedPriceActive`: "Thrown when trying to disable bounded pricing for an asset while protection is active"
- `InvalidResetThreshold`: Update comment to "at or above the trigger threshold"
- `updateProtectionState`: Update to "caching the resolved prices (both collateral and debt)"

VLD-06 Misleading Documentation For `_exceedsDeviationThreshold` Function

Category	Severity	Location	Status
Inconsistency	● Informational	contracts/DeviationBoundedOracle.sol (oracle): 735-738	● Resolved

Description

The documentation for the `_exceedsDeviationThreshold()` function is misleading and could cause confusion about the actual implementation logic.

The function comment states: "Checks if the spot price has deviated beyond the threshold from the window bounds" which suggests a simple out-of-range check. However, the actual implementation performs a more sophisticated "window spread / move from opposite extreme" check:

```
function _exceedsDeviationThreshold(
    uint256 spot,
    uint128 minPrice,
    uint128 maxPrice,
    uint256 threshold
) internal pure returns (bool) {
    uint256 upperBound = (uint256(minPrice) * (EXP_SCALE + threshold)) / EXP_SCALE;
    uint256 lowerBound = (uint256(maxPrice) * (EXP_SCALE - threshold)) / EXP_SCALE;
    return (spot > upperBound || spot < lowerBound);
}
```

This actually checks if the spot price exceeds `minPrice * (1 + threshold)` OR falls below `maxPrice * (1 - threshold)`. The wording "deviated beyond the threshold from the window bounds" could be misinterpreted as checking if spot is outside the `[minPrice, maxPrice]` range, which is not what the function does.

Recommendation

Update the function documentation to clearly explain the actual logic: "Checks if the spot price has moved beyond the threshold relative to the opposite window bound - i.e., `spot > minPrice * (1 + threshold)` OR `spot < maxPrice * (1 - threshold)`."

APPENDIX | VENUS LABS - DUAL PRICE ORACLE

Audit Scope

VenusProtocol/oracle

-  contracts/DeviationBoundedOracle.sol
-  contracts/interfaces/IDeviationBoundedOracle.sol

VenusProtocol/venus-protocol

-  contracts/Comptroller/Diamond/facets/FacetBase.sol
-  contracts/Lens/ComptrollerLens.sol
-  contracts/Tokens/VAI/VAIController.sol
-  contracts/Comptroller/ComptrollerStorage.sol
-  contracts/Comptroller/ComptrollerInterface.sol
-  contracts/Comptroller/Diamond/facets/MarketFacet.sol
-  contracts/Comptroller/Diamond/facets/PolicyFacet.sol
-  contracts/Comptroller/Diamond/facets/SetterFacet.sol
-  contracts/Comptroller/Diamond/Diamond.sol
-  contracts/Comptroller/Diamond/interfaces/ISetterFacet.sol

Finding Categories

Categories	Description
Concurrency	Concurrency findings are about issues that cause unexpected or unsafe interleaving of code executions.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.

Categories	Description
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Venus Labs - Dual Price Oracle Security Assessment | CertiK Assessed on Apr 23rd, 2026 | Copyright © CertiK

